

1일차(2)

📅 날짜	2026년 7월 2일
⚙️ 상태	완료
# 시간	3
≡ 요약	머신러닝 기초 & Amazon Comprehend 를 활용한 텍스트 AI [실습] 데이터 감성 분석 및 토픽 추출 파이프라인 구현

머신러닝 기초모델링과 피팅

Modeling: 데이터의 패턴을 함수로 매핑하는 것

Fitting: 그 함수 내부값들을 데이터에 맞게 조정

- 입력(X) → 모델 f(X) → 예측(Y') : 예측 (Y')이 정답(Y)에 가까워지도록
- 이때 얼마나 틀렸는 가를 재는 것: Loss 함수 → Loss을 줄이는 방향으로 학습
- Underfitting / Overfitting

파라미터 vs 하이퍼파라미터

→ 누가 정하느냐에 따라

	Parameter	Hyperparameter
누가 정하느냐	모델	사람
언제 정해지는지	학습(피팅) 도중 갱신	학습 시작 전 고정
예시	회귀계수(w), 절편 b, 신경망 weight	learning rate, epochs, tree depth, k-means의 k, 규제 강도

- 하이퍼파라미터 튜닝(greed search, random search, baysian optimization 등) → valid set으로 평가해야 함

혼동 행렬과 평가지표

	예측: 양성(P)	예측: 음성(N)
실제: 양성(P)	TP 맞게 잡은 양성	FN 놓친 양성 : 2종 오류
실제: 음성(N)	FP 헛경보 : 1종 오류	TN 맞게 거른 음성

Accuracy: (TP+ TN) / 전체

Precision: TP / (TP+FP)

Recall: TP / (TP+FN)

F1 score (precision, recall의 조화평균):
 $2 \cdot (P \cdot R) / (P + R)$

AUC(ROC 아래 면적): 적분

- data imabalance → accuracy 보다 Precision, Recall, F1 score 위주로
- threshold와 무관하게 종합적 성능 → AUC

Amazon Comprehend 를 활용한 텍스트 AI

[attachment:d249a9aa-56c7-4856-8c47-34e696c7f8ed:Slide-02_AWS_Comprehend_v3.pdf](#)

[실습] 데이터 감성 분석 및 토픽 추출 파이프라인 구현

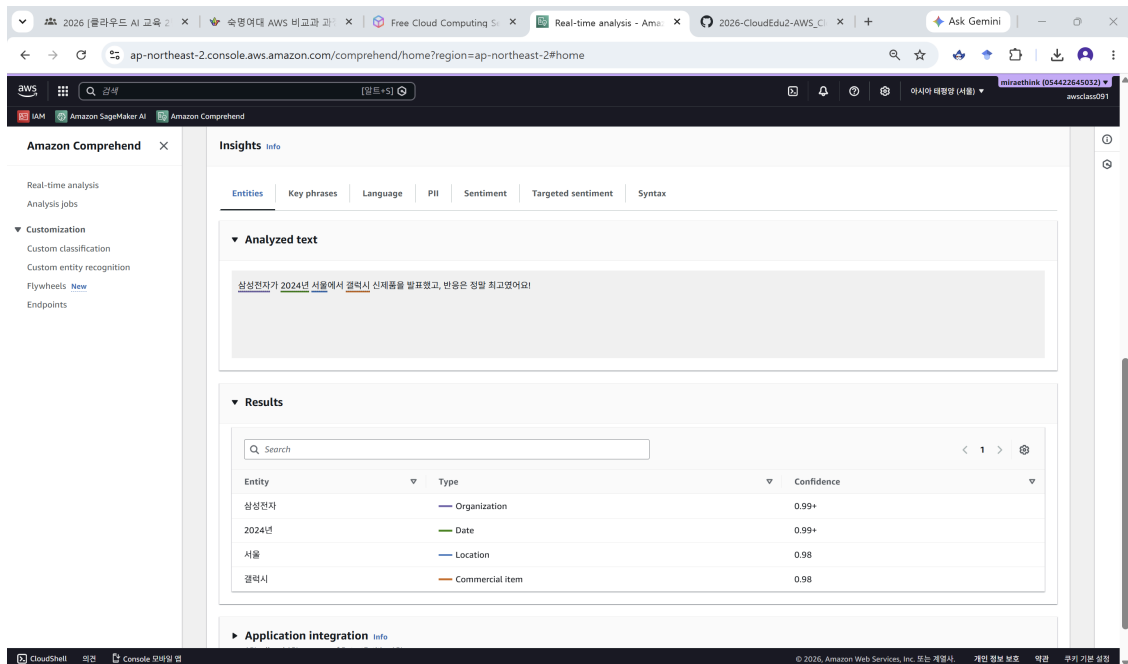
AWS console 접속

Amazon Comprehend 이동

▼ Real-time analysis

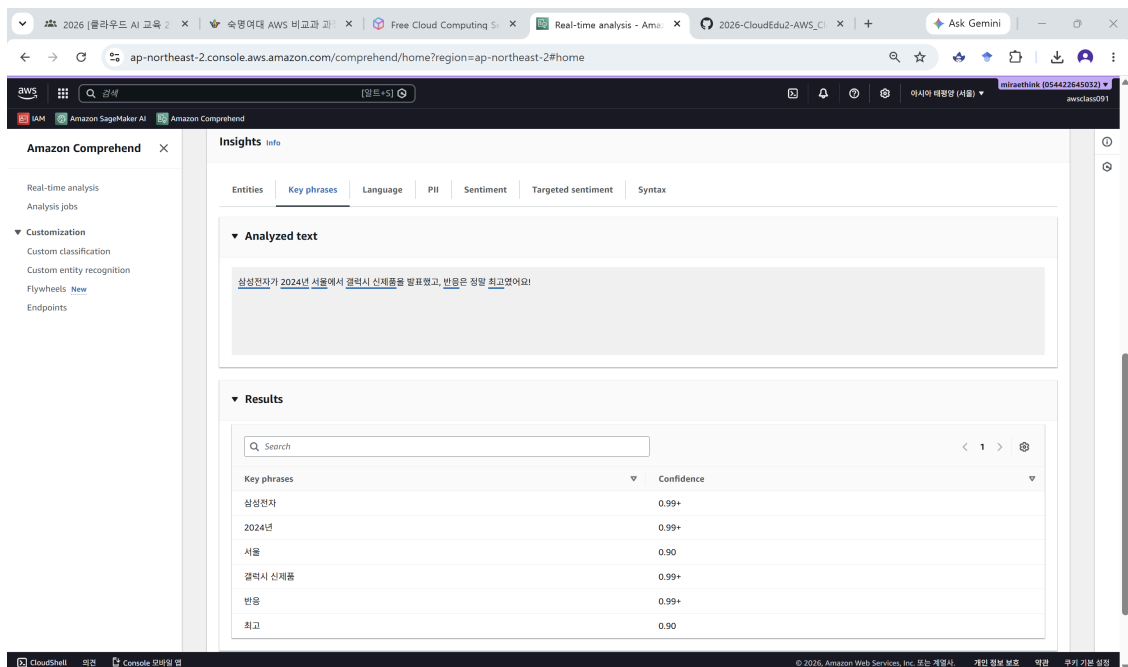
“삼성전자가 2024년 서울에서 갤럭시 신제품을 발표했고, 반응은 정말 최고였어요!” 입력

• 결과



Amazon Comprehend console showing the 'Entities' tab. The analyzed text is "삼성전자가 2024년 서울에서 갤럭시 신제품을 발표했고, 반응은 정말 최고였어요!". The results table shows the following entities:

Entity	Type	Confidence
삼성전자	Organization	0.99+
2024년	Date	0.99+
서울	Location	0.98
갤럭시	Commercial item	0.98



Amazon Comprehend console showing the 'Key phrases' tab. The analyzed text is "삼성전자가 2024년 서울에서 갤럭시 신제품을 발표했고, 반응은 정말 최고였어요!". The results table shows the following key phrases:

Key phrases	Confidence
삼성전자	0.99+
2024년	0.99+
서울	0.90
갤럭시 신제품	0.99+
반응	0.99+
최고	0.90

Amazon Comprehend

Real-time analysis
Analysis jobs

Customization
Custom classification
Custom entity recognition
Flywheels [New](#)
Endpoints

Clear text [Analyze](#)

Insights [Info](#)

Entities Key phrases **Language** PII Sentiment Targeted sentiment Syntax

▼ Analyzed text

삼성전자가 2024년 서울에서 갤럭시 신제품을 발표했고, 반응은 정말 최고였어요!

▼ Results

Language

Korean, ko
0.79 confidence

► Application integration [Info](#)
API call and API response of DetectDominantLanguage API

© 2026, Amazon Web Services, Inc. 또는 계열사. 개인 정보 보호 약관 우리 기본 설정

Amazon Comprehend

Real-time analysis
Analysis jobs

Customization
Custom classification
Custom entity recognition
Flywheels [New](#)
Endpoints

Clear text [Analyze](#)

Insights [Info](#)

Entities Key phrases Language PII **Sentiment** Targeted sentiment Syntax

▼ Analyzed text

삼성전자가 2024년 서울에서 갤럭시 신제품을 발표했고, 반응은 정말 최고였어요!

▼ Results

Sentiment

Neutral
0.00 confidence
Positive
0.99 confidence
Negative
0.00 confidence
Mixed
0.00 confidence

© 2026, Amazon Web Services, Inc. 또는 계열사. 개인 정보 보호 약관 우리 기본 설정

▼ 미션

미션 2. 개체 사냥 (한국어 · Entities 탭)

아래 뉴스 문장을 넣고, 색깔별로 개체가 몇 개씩 잡히는지 세어보세요.

2024 년 3 월, 삼성전자는 서울 강남구에서 갤럭시 S25 를 출시했고 이재용 회장이 애플과의 경쟁을 언급했다.

미션: PERSON, ORGANIZATION, LOCATION, DATE, COMMERCIAL_ITEM 을 각각 찾아 표시해 보세요. 신뢰도가 가장 낮은 개체는 무엇인가요?

• 결과

The screenshot shows the Amazon Comprehend console interface. The 'Entities' tab is selected, displaying the analyzed text and a table of results. The text is: "2024 년 3 월, 삼성전자는 서울 강남구에서 갤럭시 S25 를 출시했고 이재용 회장이 애플과의 경쟁을 언급했다." The results table lists the following entities:

Entity	Type	Confidence
2024 년 3 월	Date	0.99+
삼성전자	Organization	0.99+
서울 강남구	Location	0.98
갤럭시 S25	Commercial item	0.99+
이재용	Person	0.99+
회장	Person	0.98
애플	Organization	0.99+

미션 4. PII 숨바꼭질 (영어 · PII 탭)

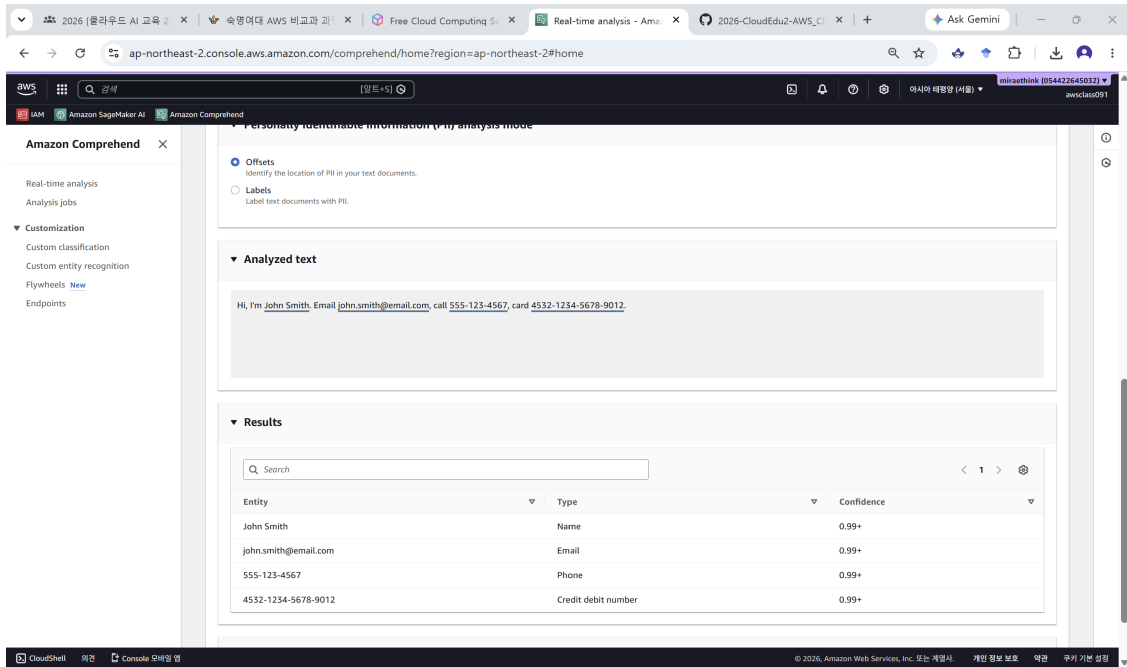
먼저 리전을 버지니아 북부(us-east-1)로 바꾸세요. 아래 문장을 넣고 Analyze 합니다.

Hi, I'm John Smith. Email john.smith@email.com, call 555-123-4567, card 4532-1234-5678-9012.

1. PII 탭에서 Offsets 모드를 봅니다 → 개인정보가 원문 어디에 있는지 위치로 표시됩니다.
2. Labels 모드로 바꿔 봅니다 → 어떤 유형(NAME, EMAIL, PHONE, CREDIT_DEBIT_NUMBER 등)이 들어있는지 라벨로 표시됩니다.

연결: 이 결과가 바로 Lab 3 의 mask_pii() 함수가 사용한 BeginOffset/EndOffset 정보입니다. 콘솔은 감지만, 마스킹 자동화는 노트북 코드로 합니다.

• 결과



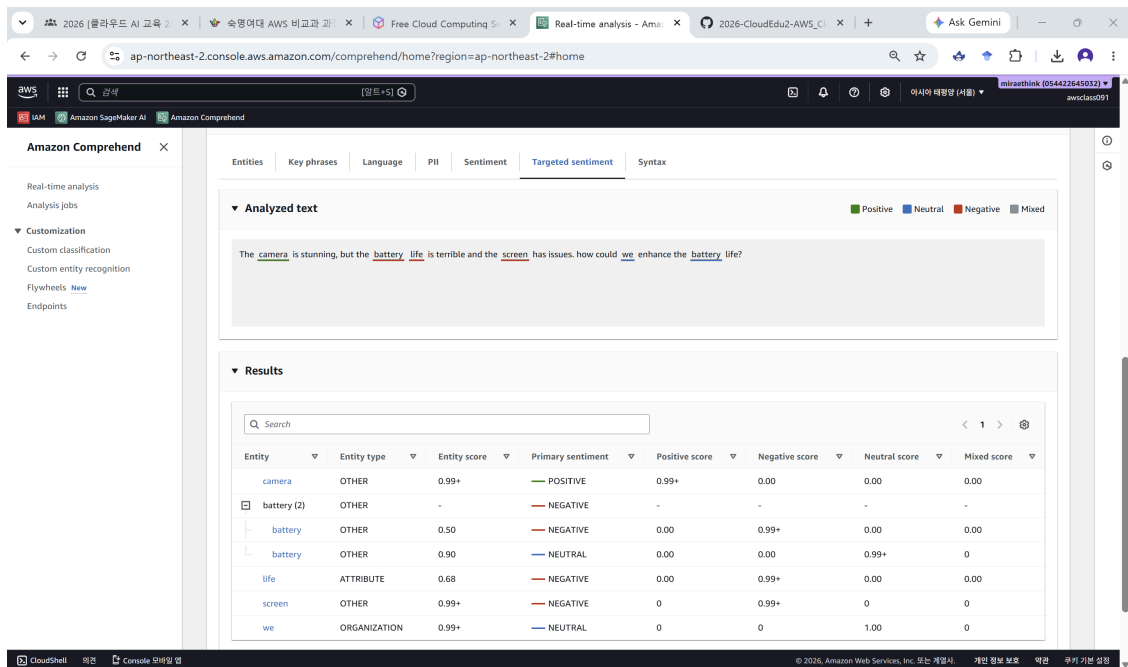
미션 5. 대상 감성 밀줄 (영어 · Targeted sentiment 탭)

한 문장 안에서 대상마다 감성이 다른 경우를 봅니다. 아래 문장을 넣고 Targeted sentiment 탭을 여세요.

The camera is stunning, but the battery life is terrible and the screen has issues.

관찰: camera / battery / screen 각각에 밀줄 색이 다르게 표시됩니다.같은 개체가 여러 번 언급되면(co-reference) 묶여서 표시됩니다.

• 결과



Amazon SageMaker AI 이동 후 SageMaker studio 열기

JupyterLab 실행 후 제공받은 파일 업로드

실습 1: 언어 감지 & 감성 분석

`comprehend.detect_dominant_language(Text=)`

– 텍스트의 언어 자동 감지

`comprehend.detect_sentiment(Text=, LanguageCode=)`

– 긍정/부정/중립/혼합 감성 분석

`comprehend.batch_detect_sentiment(TextList=, LanguageCode=)`

– 여러 문서(최대 25개)를 한 번에 분석

2026 (클라우드) x 속명여대 AWS x Free Cloud Co x Real-time anal x Jupyterlab - S x comprehend-l x 2026-CloudEd x + Ask Gemini

zwvga4a3zxwrrw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/tree/Day1(2)/comprehend-lab1-student.ipynb

File Edit View Run Kernel Git Tabs Settings Help default-20260702T110217 / awsclass091

comprehend-lab1-student.ipynb

Comprehend 클라이언트 생성 완료 (서버 ap-northeast-2)

TODO 1: detect_dominant_language — 언어 감지

다국어 텍스트 목록에서 각 문장의 언어를 자동으로 감지하세요.

```
[6]: # TODO 1: 언어 감지 API를 호출하고 결과를 출력하세요
texts = [
    '배수가 빠르고 포장도 꼼꼼했어요. 재구매 의사 있습니다.',
    'The fabric quality is amazing for the price. Highly recommend!',
    '사이즈가 작은데, 사이즈가 크네요.',
    'La couleur est un peu différente de la photo, mais je suis satisfaite.',
]

print('언어 감지 결과:')
print('-' * 55)
for text in texts:
    response = comprehend.detect_dominant_language(
        Text=text # + text - 분석할 입력 문자열 (문자열)
    )
    top = response['Languages'][0]
    lang_code = top['LanguageCode'] # + 'LanguageCode' - 감지된 언어 코드 (ko=한국어, en=영어, ja=일본어, fr=프랑스어 등)
    score = top['Score'] # + 'Score' - 감지 신뢰도 (0~1, 1에 가까울수록 확실)
    print(f' [{lang_code}] {score:.3f} | {text[:30]}')
```

언어 감지 결과:

```
[ko] 0.793 배수가 빠르고 포장도 꼼꼼했어요. 재구매 의사 있습니다
[en] 0.974 The fabric quality is amazing
[ja] 1.000 사이즈가 작은데, 사이즈가 크네요.
[fr] 0.997 La couleur est un peu différen
```

TODO 2: detect_sentiment — 단일 문서 감성 분석

2026 (클라우드) x 속명여대 AWS x Free Cloud Co x Real-time anal x Jupyterlab - S x comprehend-l x 2026-CloudEd x + Ask Gemini

zwvga4a3zxwrrw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/tree/Day1(2)/comprehend-lab1-student.ipynb

File Edit View Run Kernel Git Tabs Settings Help default-20260702T110217 / awsclass091

comprehend-lab1-student.ipynb

TODO 2: detect_sentiment — 단일 문서 감성 분석

리뷰 텍스트의 감성과 각 감성 점수를 출력하세요.

```
[8]: # TODO 2: 감성 분석 API를 호출하고 결과를 출력하세요
reviews = [
    '이 제품 정말 최고예요! 배송도 빠르고 품질도 훌륭합니다.',
    '완전 실망입니다. 사진과 너무 다르고 품질이 형편없어요.',
    '그냥 평범한 제품이에요. 나쁘지도 좋지도 않습니다.',
    '좋은 점도 있고 아쉬운 점도 있어요. 가격 대비는 괜찮네요.',
]

results = []
for review in reviews:
    response = comprehend.detect_sentiment(
        Text=review, # + review - 감성을 분석할 리뷰 텍스트
        LanguageCode='ko' # + 'ko' - 입력 언어를 명시 (한국어). 언어를 알 때는 직접 지정이 정확함
    )
    sentiment = response['Sentiment'] # + 'Sentiment' - 대표 감성 (POSITIVE/NEGATIVE/NEUTRAL/MIXED 중 하나)
    scores = response['SentimentScore'] # + 'SentimentScore' - 감성별 확률 dict (Positive, Negative, Neutral, Mixed) (합계 1)
    results.append({'text': review[:25], 'sentiment': sentiment, 'scores': scores})
    print(f' [{sentiment}] | {review[:30]}')
```

print('\n상세 점수 (첫 번째 리뷰):')

```
for k, v in results[0]['scores'].items():
    print(f' {k:12}: {v:.4f}')
```

상세 점수 (첫 번째 리뷰):

```
Positive : 0.9999
Negative : 0.0001
Neutral : 0.0001
Mixed : 0.0000
```

이제 코드 실행해서 점수 (1가지)

comprehend-lab1-student.ipynb

TODO 3: batch_detect_sentiment — 배치 처리

여러 문서를 한 번의 API 호출로 처리하고 결과를 DataFrame으로 정리하세요.

```
[11]: # TODO 3: batch_detect_sentiment 여러 문서를 한 번에 분석하세요
batch_texts = [
    '사이즈가 딱 맞고 색상도 화면이랑 똑같아요.',
    '배출이 일주일 넘게 걸렸어요. 너무 느립니다.',
    '가성비 최고입니다. 친구에게도 추천했어요.',
    '주요한 색상과 다른 색이 있어요. 교환 요청합니다.',
    '품질 대비 가격이 합리적이에요. 만족합니다.'
]

response = comprehend.batch_detect_sentiment(
    TextList=batch_texts, # batch_texts: 한 번에 분석할 문서 리스트 (배치당 최대 25건)
    LanguageCode='ko' # 'ko': 배치 전체에 공통 적용되는 언어
)

rows = []
for item in response['ResultList']: # 'ResultList': 문서별 결과 리스트. item['Index']로 원본 순서와 매칭
    idx = item['Index']
    sent = item['Sentiment']
    pos = item['SentimentScore']['Positive']
    neg = item['SentimentScore']['Negative']
    rows.append({'텍스트': batch_texts[idx], '감성': sent,
                'Positive': round(pos,3), 'Negative': round(neg,3)})

df = pd.DataFrame(rows)
print(df.to_string(index=False))
```

텍스트	감성	Positive	Negative
사이즈가 딱 맞고 색상도 화면이랑 똑같아요.	POSITIVE	0.994	0.002
배출이 일주일 넘게 걸렸어요. 너무 느립니다.	NEGATIVE	0.000	1.000
가성비 최고입니다. 친구에게도 추천했어요.	POSITIVE	1.000	0.000
주요한 색상과 다른 색이 있어요. 교환 요청합니다.	NEGATIVE	0.013	0.958
품질 대비 가격이 합리적이에요. 만족합니다.	POSITIVE	1.000	0.000

실습 2: 개체 인식(NER) & 핵심 문구 추출

`comprehend.detect_entities(Text=, LanguageCode=)`

- 인물·조직·장소·날짜·금액·제품명 등 개체 추출

`comprehend.detect_key_phrases(Text=, LanguageCode=)`

- 핵심 명사구 (주제) 추출

comprehend-lab2-student.ipynb

TODO 1: detect_entities — 개체 인식

뉴스 기사 텍스트에서 개체를 추출하고 유형별로 분류하세요.

```
[2]: # TODO 1: detect_entities API를 호출하고 결과를 출력하세요
news_text = """
2024년 3월 15일, 삼성전자는 서울 강남구 삼성 서초사옥에서 갤럭시 S25를 공식 출시했습니다.
CEO 이재용 부회장이 직접 발표를 진행했으며, 애플의 아이폰 15와의 경쟁에서
우위를 점할 것으로 기대된다고 밝혔습니다. 출시 행사에는 약 500명의 기자가 참석했습니다.
"""

response = comprehend.detect_entities(
    Text=news_text, # news_text: 개체를 추출할 원문
    LanguageCode='ko' # 'ko': 입력 언어 (한국어)
)

entities = response['Entities'] # 'Entities': 감지된 개체 리스트 (각 원소는 Text/Type/Score/위치 포함)
print(f'감지된 개체: {len(entities)}개')
print('-' * 60)
for e in entities:
    print(f"['{e['Type']}<20'] ({e['Text']}<20) (신뢰도: {e['Score']:.3f})" # 'Type': 개체 유형 (PERSON/LOCATION/ORGANIZATION/DATE/QUANTITY 등)

감지된 개체: 10개
-----
[DATE]                ] 2024년 3월 15일                (신뢰도: 0.987)
[ORGANIZATION]        ] 삼성전자                (신뢰도: 0.999)
[LOCATION]              ] 서울 강남구 삼성 서초사옥    (신뢰도: 0.921)
[COMMERCIAL_ITEM]      ] 갤럭시 S25            (신뢰도: 0.995)
[PERSON]               ] CEO                (신뢰도: 0.989)
[PERSON]               ] 이재용                (신뢰도: 0.987)
[PERSON]               ] 부회장                (신뢰도: 0.923)
[ORGANIZATION]         ] 애플                (신뢰도: 0.996)
[COMMERCIAL_ITEM]      ] 아이폰 15            (신뢰도: 0.998)
[QUANTITY]             ] 약 500명의 기자        (신뢰도: 0.866)
```

comprehend-lab2-student.ipynb

TODO 2: detect_key_phrases — 핵심 문구 추출

텍스트에서 핵심 명사구를 추출하고 신뢰도 순으로 정렬하세요.

```
[4]: # TODO 2: detect_key_phrases API를 호출하고 결과를 정리하세요
article = """
인공지는 기술의 급속한 발전으로 의료 분야에서 혁신적인 변화가 일어나고 있습니다.
디지털 기반 진단 시스템은 암 조기 발견 정확도를 98% 이상으로 높였으며,
자면어치리 기술을 활용한 의료 맞춤형 환자 상담 서비스를 24시간 제공합니다.
"""

response = comprehend.detect_key_phrases(
    Text=article, # + article - 핵심 문구를 추출할 원문
    LanguageCode='ko' # + 'ko' - 입력 언어 (한국어)
)

phrases = response['KeyPhrases'] # + 'KeyPhrases' - 추출된 핵심 명사구 리스트 (Text/Score_포함)

# 신뢰도 기준 내림차순 정렬
phrases_sorted = sorted(phrases, key=lambda x: x['Score'], reverse=True) # + 'Score' - 핵심문구 신뢰도(0-1) 기준 내림차순 정렬

print(f'추출된 핵심 문구: {len(phrases)}개')
print('-' * 50)
for p in phrases_sorted[:10]:
    print(f"  {p['Score']:.3f}  {p['Text']}")

추출된 핵심 문구: 10개
-----
1.000 자연어처리 기술
1.000 디지털 기반 진단 시스템
0.999 의료 분야
0.999 의료 맞춤
0.999 환자 상담 서비스
0.999 암 조기 발견 정확도
0.990 24시간
0.976 혁신적인 변화
0.944 98%
0.939 인공지는 기술의 급속한 발전
```

실습 3: 대상 감성 분석 & PII 감지

`comprehend.detect_targeted_sentiment(Text=, LanguageCode=)`

– 대상 감성 분석 (개체별 감성)

`comprehend.detect_pii_entities(Text=, LanguageCode=)`

– PII 감지

comprehend-lab3-student.ipynb

TODO 1: detect_targeted_sentiment — 개체별 감성 분석

제품 리뷰에서 각 개체(배터리, 카메라, 디자인 등)별 감성을 개별 분석하세요.

```
[2]: # TODO 1: detect_targeted_sentiment API를 호출하고 개체별 감성을 출력하세요
review = (
    "The camera quality is absolutely stunning and takes great photos. "
    "However, the battery life is terrible and drains too fast. "
    "The design looks premium but the screen has some issues."
)

response = comprehend.detect_targeted_sentiment(
    Text=review, # + review - 개체별 감성을 분석할 리뷰
    LanguageCode='en' # + 'en' - 이 기능(대상감성-PII)은 영어만 지원 → us-east-1 사용
)

entities = response['Entities'] # + 'Entities' - 개체 리스트. 각 개체는 Mentions(언급)와 감성 정보 포함
print(f'분석된 개체: {len(entities)}개')
print('-' * 60)
for e in entities:
    mention = e['Mentions'][0]
    text = mention['Text']
    sent = mention['SentimentScore']['Sentiment'] # + 'Sentiment' - 해당 개체에 대한 감성(전체 문장이 아닌 개체 단위)
    score = mention['SentimentScore']['Score']
    dominant = max(score, key=score.get)
    print(f"  {text}<20> → {sent:<12}> (최고: {dominant} {score[dominant]:.3f})")

분석된 개체: 6개
-----
camera          → POSITIVE   (최고: Positive 1.000)
quality         → POSITIVE   (최고: Positive 1.000)
battery         → NEGATIVE   (최고: Negative 1.000)
design           → POSITIVE   (최고: Positive 1.000)
screen          → NEGATIVE   (최고: Negative 1.000)
some            → NEUTRAL    (최고: Neutral 1.000)
```

TODO 2: detect_pii_entities — PII 감지

텍스트에서 개인식별정보(이름, 이메일, 전화번호, 주소 등)를 감지하세요.

```
[4]: # TODO 2: detect_pii_entities API를 호출하고 결과를 출력하세요

pii_text = (
    "Hello, my name is John Smith. You can reach me at john.smith@email.com "
    "or call me at 555-123-4567. My address is 123 Main Street, New York, NY 10001. "
    "My credit card number is 4532-1234-5678-9012."
)

response = comprehend.detect_pii_entities(
    Text=pii_text, # + pii_text - 개인정보를 감지할 원문
    LanguageCode='en' # + 'en' - 이 기능(대상감성-PII)은 영어만 지원 + us-east-1 사용
)

pii_entities = response['Entities'] # + 'Entities' - 감지된 PII 리스트 (Type: 위치 offset 포함, 원문 길은 직접 주지 않음)
print(f'감지된 PII: {len(pii_entities)}개')
print('-' * 60)
for p_ent in pii_entities:
    start = p_ent['BeginOffset'] # + 'BeginOffset' - 원문에서 PII가 시작하는 문자 위치 (인덱스)
    end = p_ent['EndOffset'] # + 'EndOffset' - 원문에서 PII가 끝나는 문자 위치 + text[start:end]로 값 복원
    text = pii_text[start:end]
    print(f'[{p_ent["Type"]}<200)] {text}<300) (신뢰도: {p_ent["Score"]:.3f})')

감지된 PII: 5개
-----
[NAME] John Smith (신뢰도: 1.000)
[EMAIL] john.smith@email.com (신뢰도: 1.000)
[PHONE] 555-123-4567 (신뢰도: 0.994)
[ADDRESS] 123 Main Street, New York, NY 10001 (신뢰도: 0.999)
[CREDIT_DEBIT_NUMBER] 4532-1234-5678-9012 (신뢰도: 1.000)
```

TODO 3: PII 마스킹 함수 구현

감지된 PII를 ***로 치환하는 마스킹 함수를 완성하세요.

```
[6]: # TODO 3: PII를 마스킹하는 함수를 완성하세요

def mask_pii(text, pii_entities):
    """PII 위치를 위에서부터 치환하여 offset 오염 방지"""
    sorted_entities = sorted(pii_entities,
                              key=lambda x: x['BeginOffset'], # + 'BeginOffset' - 시작 위치 기준 정렬
                              reverse=True) # + True - 뒤(큰 offset)에서부터 치환 + 앞 글자 위치가 밀리지 않음

    masked = text
    for ent in sorted_entities:
        start = ent['BeginOffset'] # + 'BeginOffset' - 치환 시작 위치
        end = ent['EndOffset'] # + 'EndOffset' - 치환 끝 위치
        marker = f"[{ent['Type']}]"
        masked = masked[:start] + marker + masked[end:]

    return masked

masked_text = mask_pii(pii_text, pii_entities)
print('원본:')
print(pii_text)
print('\n마스킹 결과:')
print(masked_text)

원본:
Hello, my name is John Smith. You can reach me at john.smith@email.com or call me at 555-123-4567. My address is 123 Main Street, New York, NY 10001. My credit card number is 4532-1234-5678-9012.

마스킹 결과:
Hello, my name is [NAME]. You can reach me at [EMAIL] or call me at [PHONE]. My address is [ADDRESS]. My credit card number is [CREDIT_DEBIT_NUMBER].
```

[보너스] PII 자동 마스킹 데모 (위젯 없이 실행)

실습 4: 구문 분석 & 종합 텍스트 분석 파이프라인

`comprehend.detect_syntax(Text=, LanguageCode=)`
 - 구문 분석 (품사 태깅)

comprehend-lab4-student.ipynb

TODO 1: detect_syntax — 품사 태깅

텍스트를 형태소 단위로 분석하고 품사 분포를 시각화하세요.

```
[2]: # TODO 1: detect_syntax API를 호출하고 품사 분포를 출력하세요
syntax_text = (
    "Amazon Web Services provides reliable cloud computing solutions "
    "that help businesses scale quickly and efficiently."
)

response = comprehend.detect_syntax(
    Text=syntax_text, # 품사 분석할 원문
    LanguageCode='en' # 'en' - 구문 분석(detect_syntax)은 영어 등 일부 언어만 지원
)

tokens = response['SyntaxTokens'] # 'SyntaxTokens' - 토큰(단어) 별 분석 결과 리스트 (Text/PartOfSpeech 포함)
print(f'토큰 수: {len(tokens)}개')
print(' ' * 50)
for tok in tokens:
    pos = tok['PartOfSpeech']['Tag'] # 'Tag' - 품사 태그 (NOUN명사/VERB동사/ADJ형용사/PROPN고유명사 등)
    conf = tok['PartOfSpeech']['Score'] # 'Score' - 품사 판정 신뢰도 (0~1)
    print(f' {tok['Text']}<15) {pos:<8) {(conf:.3f)})'

-----
토큰 수: 16개
-----
Amazon      PROPN   (1.000)
Web         PROPN   (1.000)
Services    PROPN   (1.000)
provides    VERB     (1.000)
reliable    ADJ      (1.000)
cloud       NOUN     (1.000)
computing   NOUN     (0.508)
solutions   NOUN     (1.000)
that        PRON     (1.000)
help        VERB     (1.000)
businesses  NOUN     (1.000)
scale       VERB     (1.000)
quickly     ADV      (1.000)
and         CCONJ    (1.000)
```

```
def analyze_text(text, lang='ko'):
    result = {'text': text}
    # 1) 언어 감지 detected = comprehend.detect_dominant_language(Text=____)
    _['Languages'][0]['LanguageCode'] # ← text
    # 2) 감성 s = comprehend.detect_sentiment(Text=____, LanguageCode=____)
    _ # ← text, detected result['sentiment'] = s['Sentiment']
    # 3) 개체
    # 4) 핵심문구 ... (동일 패턴) return result
```

TODO 2: 종합 분석 파이프라인 구현

언어 감지 → 감성 분석 → 개체 인식 → 핵심 문구 추출을 하나의 함수로 통합하세요.

```
[4]: # TODO 2: analyze_text() 통합 파이프라인 함수를 완성하세요
def analyze_text(text, lang='ko'):
    """Comprehend 종합 분석 파이프라인"""
    result = {'text': text, 'language': lang}

    # 1단계: 언어 감지
    lang_resp = comprehend.detect_dominant_language(text) # = text - 언어를 자동 감지함
    detected_lang = lang_resp['languages'][0]['languageCode'] # = 'LanguageCode' - 가장 유력한 언어 코드 (아무 단계에 제지됨)
    result['detected_language'] = detected_lang

    # 2단계: 감성 분석
    sent_resp = comprehend.detect_sentiment(
        Text=text, LanguageCode=detected_lang # = text, detected_lang - 분석 원문과 일치한 언어
    )
    result['sentiment'] = sent_resp['Sentiment'] # = 'Sentiment' - 대표 감성 (POSITIVE/NEGATIVE/NEUTRAL/MIXED)
    result['sentiment_scores'] = sent_resp['SentimentScore'] # = 'SentimentScore' - 감성별 확률 dict

    # 3단계: 개체 인식
    ent_resp = comprehend.detect_entities(
        Text=text, LanguageCode=detected_lang # = text, detected_lang - 분석 원문과 일치한 언어
    )
    result['entities'] = [
        {'text': e['Text'], 'type': e['Type'], 'score': round(e['Score'], 3)}
        for e in ent_resp['Entities']
    ]
    # = 'Entities' - 감지된 개체 리스트

    # 4단계: 핵심 문구
    kp_resp = comprehend.detect_key_phrases(
        Text=text, LanguageCode=detected_lang # = text, detected_lang - 분석 원문과 일치한 언어
    )
    result['key_phrases'] = [p['Text'] for p in kp_resp['KeyPhrases']] # = 'KeyPhrases' - 추출된 핵심 문구 리스트

    return result

# 테스트
text = '상정선거 2024년 서울에서 AI 반도체 산업을 발표했습니다.'
out = analyze_text(text)
print(f"언어: {out['detected_language']}")
print(f"감성: {out['sentiment']}")
print(f"개체: {out['entities']}")
print(f"핵심문구: {out['key_phrases']}")

언어: ko
감성: NEUTRAL
개체: [{"text": '상정선거', 'type': 'ORGANIZATION', 'score': 0.999}, {"text": '2024년', 'type': 'DATE', 'score': 0.996}, {"text": '서울', 'type': 'LOCATION', 'score': 0.985}, {"text": 'AI', 'type': 'OTHER', 'score': 0.591}]
핵심문구: ['상정선거', '2024년', '서울', 'AI 반도체 산업']
```

TODO 3: 배치 파이프라인 & 결과 저장

TODO 3: 배치 파이프라인 & 결과 저장

여러 뉴스 헤드라인을 파이프라인으로 처리하고 결과를 JSON과 CSV로 저장하세요.

```
[5]: # TODO 3: 여러 텍스트를 파이프라인으로 처리하고 결과를 저장하세요
headlines = [
    '현대차, 전기차 신모델 출시로 대승리와 본격 경쟁 선언',
    '코스피 3,000선 돌파, 외국인 투자자 대규모 순매수',
    'BTS 월드투어 티켓 매진, 팬들 아쉬움 토로',
    '기상청, 이번 주말 전국 폭설 예보'
]

all_results = []
for i, h in enumerate(headlines):
    print(f"처리 중 [{i+1}/{len(headlines)}]: {h[:25]}...")
    r = analyze_text(h) # = analyze_text - 앞에서 만든 통합 파이프라인 함수 호출
    all_results.append(r)

# JSON 저장
with open('./lab04_output/results.json', 'w') as f: # = 'w' - 쓰기 모드 (write)
    json.dump(all_results, f, ensure_ascii=False, indent=2) # = all_results - 저장할 전체 분석 결과 리스트
    print(f"JSON results.json 저장 완료")

# CSV 저장
df = pd.DataFrame([
    {'헤드라인': r['text'][:30],
     '감성': r['sentiment'],
     '개체': '\n'.join(r['entities']),
     '핵심문구': '\n'.join(r['key_phrases'])[:30]}
    for r in all_results])
df.to_csv('./lab04_output/results.csv', index=False, encoding='utf-8-sig')
print(f"CSV results.csv 저장 완료")
print(df.to_string(index=False))

처리 중 [1/4]: 현대차, 전기차 신모델 출시로 대승리와 본격 경쟁 선언...
처리 중 [2/4]: 코스피 3,000선 돌파, 외국인 투자자 대규모 순매수...
처리 중 [3/4]: BTS 월드투어 티켓 매진, 팬들 아쉬움 토로...
처리 중 [4/4]: 기상청, 이번 주말 전국 폭설 예보...

results.json 저장 완료
results.csv 저장 완료
```

헤드라인	감성	개체	핵심문구
현대차, 전기차 신모델 출시로 대승리와 본격 경쟁 선언	NEUTRAL	현대차, 전기차 신모델 출시, 대승리와	
코스피 3,000선 돌파, 외국인 투자자 대규모 순매수	NEUTRAL	코스피 3,000선 돌파, 외국인 투자자 대규모 순매수	
BTS 월드투어 티켓 매진, 팬들 아쉬움 토로	NEUTRAL	BTS 월드투어 티켓 매진, 팬들, 아쉬움	
기상청, 이번 주말 전국 폭설 예보	NEUTRAL	기상청, 이번 주말 전국 폭설 예보	

실습 5: 토픽 모델링 & 비지도 학습(군집) 개념 체형

vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b", stop_words=STOP_WORDS)

X = vectorizer.fit_transform()

- 문서 벡터화(TF-IDF): 텍스트 → 숫자

km = KMeans(n_clusters=, random_state=42, n_init=10)

```
labels = km.fit_predict( )
- KMeans 군집화: 라벨 없이 토픽 묶기

new_X = vectorizer.transform( )
pred = km.predict( )
- 새 리뷰의 토픽 예측
```

The screenshot shows a JupyterLab interface with a notebook titled 'comprehend-lab5-student.ipynb'. The code in the notebook includes:

```
print("참고: Comprehend 토픽 모델링은 S3 + 빅데이터 클러스터로 동작합니다. (수집 분 소요).")
print("아래에서는 같은 개념(비지도 군집)을 즉시 체험합니다.")

참고: Comprehend 토픽 모델링은 S3 + 빅데이터 클러스터로 동작합니다. (수집 분 소요).
아래에서는 같은 개념(비지도 군집)을 즉시 체험합니다.
```

TODO 1: 문서 벡터화 (텍스트 → 숫자)

군집을 하려면 먼저 문장을 숫자(벡터)로 바꿔야 합니다. TF-IDF 는 각 문서에서 '자주 특징적으로 등장하는 단어에 가중치를 찍어 벡터로 만듭니다.

```
[4]: # TODO 1: 샘플 데이터를 TF-IDF로 벡터화해줘요
from sklearn.feature_extraction.text import TfidfVectorizer

# 토픽이 모였을 샘플 라벨 (배출 / 품질 / 가격 / 판매 / 상담)
sample_docs = [
    '배출 속도가 느리고 배송이 일주일 걸렸어요',
    '배출 박스가 피그라져서 왔어요',
    '배송 빨라서 좋고 당일 배송 편리해요',
    '배송 기사님 친절하고 배송 상태 좋아요',
    '제품 품질이 사진과 달라 품질 실망이에요',
    '원단 품질 훌륭하고 품질 튼튼해요',
    '품질 균형 잡가지고 품질 별로예요',
    '마감 품질 깔끔하고 품질 만족해요',
    '가격 비싸고 환불 원하는데 환불 안돼요',
    '가격 합리적이고 가격 가성비 좋아요',
    '환불 절차 복잡하고 환불 느려요',
    '환불 가격 저당해서 가격 만족해요',
    '상담 직원어 불친절하고 상담 별로예요',
    '상담 답변 빠르고 상담 친절해요',
    '상담 안돼 오래 걸리고 상담 불만해요',
    '고객센터 상담 만족하고 상담 좋아요',
]

# 일반적인 단어(불용어)는 토픽 구분에 도움이 안 되므로 제거합니다
STOPWORDS = ['너무', '정말', '아주', '별로', '그냥', '좋아요', '좋고', '잘했어요', '잘하고',
    '맛있어요', '만족해요', '편리해요', '친절해요', '빠른', '빠르네요', '최고입니다',
    '저렴해서', '빠르고', '느리고', '느려요', '늦었고', '잘못하고', '불만하고',
    '튼튼해요', '불친절하고', '별로예요', '실망이에요', '왕가지고', '피그라져서']

vectorizer = TfidfVectorizer(token_pattern='(?u)\\b\\w\\b', stop_words=STOPWORDS)
X = vectorizer.fit_transform(sample_docs) # = sample_docs : 학습용 문서 리스트, fit_transform: 단어사진 학습=벡터변환, 문서
print("문서 수:", X.shape[0], " | 단어(특성) 수:", X.shape[1])

문서 수: 16 | 단어(특성) 수: 32
```

TODO 2: KMeans 군집화 — 라벨 없이 토픽 묶기

KMeans 는 비슷한 벡터끼리 K개의 그룹(군집)으로 묶는 대표적인 비지도 학습입니다. 여기서는 토픽 4개(배출 / 품질 / 가격 / 판매)를 기대하고 K=4로 묶어봅니다.

The screenshot shows the same JupyterLab interface with the notebook 'comprehend-lab5-student.ipynb'. The code continues with KMeans clustering:

```
vectorizer = TfidfVectorizer(token_pattern='(?u)\\b\\w\\b', stop_words=STOPWORDS)
X = vectorizer.fit_transform(sample_docs) # = sample_docs : 학습용 문서 리스트, fit_transform: 단어사진 학습=벡터변환, 문서
print("문서 수:", X.shape[0], " | 단어(특성) 수:", X.shape[1])

문서 수: 16 | 단어(특성) 수: 32
```

TODO 2: KMeans 군집화 — 라벨 없이 토픽 묶기

KMeans 는 비슷한 벡터끼리 K개의 그룹(군집)으로 묶는 대표적인 비지도 학습입니다. 여기서는 토픽 4개(배출 / 품질 / 가격 / 판매)를 기대하고 K=4로 묶어봅니다.

```
[5]: # TODO 2: KMeans로 문서를 4개 토픽으로 군집화해줘요
from sklearn.cluster import KMeans

N_TOPICS = 4
km = KMeans(n_clusters=N_TOPICS, random_state=42, n_init=10) # = N_TOPICS : 토픽 군집(토픽) 개수=4, random_state: 결과 재현을 고정함
labels = km.fit_predict(X) # = X : TF-IDF 벡터 배열, 군집 학습 후 문서별 토픽 번호 반환

print("문서별 자동 배정 토픽 (라벨 없이 묶인 결과):")
print("-" * 50)
for i, doc in enumerate(sample_docs):
    print(f" [토픽 {labels[i]}] | {doc}")

문서별 자동 배정 토픽 (라벨 없이 묶인 결과):
-----
[토픽 0] 배출 속도가 느리고 배송이 일주일 걸렸어요
[토픽 0] 배출 박스가 피그라져서 왔어요
[토픽 0] 배송 빨라서 좋고 당일 배송 편리해요
[토픽 0] 배송 기사님 친절하고 배송 상태 좋아요
[토픽 0] 제품 품질이 사진과 달라 품질 실망이에요
[토픽 0] 원단 품질 훌륭하고 품질 튼튼해요
[토픽 0] 품질 균형 잡가지고 품질 별로예요
[토픽 0] 마감 품질 깔끔하고 품질 만족해요
[토픽 0] 가격 비싸고 환불 원하는데 환불 안돼요
[토픽 0] 가격 합리적이고 가격 가성비 좋아요
[토픽 0] 환불 절차 복잡하고 환불 느려요
[토픽 0] 환불 가격 저당해서 가격 만족해요
[토픽 1] 상담 직원어 불친절하고 상담 별로예요
[토픽 1] 상담 답변 빠르고 상담 친절해요
[토픽 1] 상담 안돼 오래 걸리고 상담 불만해요
[토픽 1] 고객센터 상담 만족하고 상담 좋아요

[6]: # [해결 코드] 토픽별 대표 단어 + 군집 크기 시각화
import numpy as np
terms = vectorizer.get_feature_names_out()

print(" 토픽별 대표 단어 (군집 중심에서 가중치 높은 단어):")
for c in range(N_TOPICS):
    print(f"토픽 {c}의 대표 단어: {terms[np.argsort(-km.cluster_centers_[c].sum(axis=0))[:10]]}")
```

2026 (클라우드) | 속명여대 AWS | Free Cloud Co | Real-time anal | Jupyterlab - S | comprehend-l | 2026-CloudEd | Ask Gemini

zwgba4a3zxwrrw.studio.ap-northeast-2.sagemaker.aws/jupyterlab/default/lab/Day1(2)/comprehend-lab5-student.ipynb

File Edit View Run Kernel Git Tabs Settings Help

comprehend-lab5-student.ipynb

Launcher

comprehend-lab5-student.ipynb

Markdown

Notebook Cluster Python 3 (pykernel)

+

Day1(2) /

Name	Modified
lab04_output	2s ago
comprehend-lab1...	56s ago
comprehend-lab2...	56s ago
comprehend-lab3...	56s ago
comprehend-lab4...	1s ago
comprehend-lab5...	5s ago
comprehend-lab6...	49m ago
lab02_result.csv	9m ago

토픽 0 토픽 1 토픽 2 토픽 3

TODO 3: 새 리뷰는 어느 토픽일까요?

학습된 군집 모델로 새 리뷰가 어느 토픽에 속하는지 예측해봅시다.

```
[*]: # TODO 3: 새 리뷰를 기존 vectorizer로 변환하고 토픽을 예측하세요

new_reviews = [
    '배웅 상자가 또 도착해서 좋아요',
    '항복 절차가 복잡하고 가격도 비싸요',
    '상당 직원이 빠르게 답변했어요',
]

new_X = vectorizer.transform(new_reviews) # ~ new_reviews - 새 문서, 기존 사전으로 transform(fit, 군집) 해야 동일 기준 적용
pred = km.predict(new_X) # ~ new_X - 변환된 새 행들을 학습된 모델로 토픽 예측

for review, topic in zip(new_reviews, pred):
    print(f' [토픽 {topic}] {review}')
```

[토픽 0] 배웅 상자가 또 도착해서 좋아요
[토픽 3] 항복 절차가 복잡하고 가격도 비싸요
[토픽 1] 상당 직원이 빠르게 답변했어요

실무로 연결하기

오늘 체험한 '라벨 없이 무기가 실무에서는 이렇게 쓰입니다.

수천 건 리뷰 VOC → 토픽 모델링(군집) → 토픽별 비중 추이 → 대시보드

- 신상품 출시 후 어떤 주제(배웅/항복/가격)의 언급이 급증하는지 자동 보직
- 라벨을 미리 만들 수 없는 새로운 이슈도 군집으로 드러남 (지도학습의 한계 보완)
- 대량 장기 분석은 Comprehend 관리형 토픽 모델링(비동기 집)으로 자동화
- 군집 결과에 감성 분석(Lab 1)을 결합하면 '어떤 주제가 부정적인지'까지 파악

Simple 6 Python 3 (pykernel) | Idle Initialized (additional servers needed) Instance MEM 78%

Cookie Preferences Mode: Command Ln 1, Col 1 comprehend-lab5-student.ipynb 4